EXPLORATORY

Time, scale, and emergent computation

ar091 · 18 August 2026

Contents

1. Computation through time

A conventional feedforward ANN neuron transforms its current input into an output value. An SNN neuron maintains private state through time and communicates selected state transitions as discrete spike events. The important distinction is not continuous computation against discrete computation. Physical neuron models may evolve continuously, while simulations usually advance in discrete steps. The important distinction is persistent temporal state combined with event-based communication.

This temporal structure supports three broad capacities. Temporal memory lets previous inputs affect present computation. Temporal coding lets information reside in spike order, latency, phase, or synchrony rather than only activation magnitude. Collective dynamics let recurrent events form rhythms, attractors, and coordinated network states.

Recurrent ANNs can emulate these capacities; temporal state and event communication are simply native to SNNs.

2. The ANN-like regime

An SNN can be made to behave approximately like an ANN when spike activity is compressed into a scalar rate or count. In that regime, private state acts mainly as an accumulator. Inputs change the state, threshold crossings turn accumulated magnitude into spikes, and a decoder reconstructs an approximate continuous activation.

This approximation requires a constrained operating region: stable and approximately monotonic firing rates, controlled leak and reset, suitable threshold and weight scaling, enough observation time, and little dependence on exact spike timing. It is one use of an SNN, not its definition. The broader system can retain information that rate decoding deliberately discards.

3. Computation through scale

A large network differs from a small one not merely by containing more computation or more independent subnetworks. Its important advantage is integrative scale: representations can interact, constrain one another, and participate in shared computation across the system.

Integrative scale supports abstraction, composition, and integration. Abstraction compresses many cases into reusable concepts. Composition combines those concepts into structured situations and multi-step computations. Integration coordinates specialised processes into coherent models, predictions, and plans.

These capacities enable generalisation, internal simulation, and model-guided behaviour. Scale changes possible coordinated dynamics, not merely available calculation.

4. Emergence

Integrative scale is closely related to emergence. Emergent behaviour is a system-level pattern produced by interactions among components but not attributable to any component in isolation. No neuron contains a belief, just as no water molecule contains a whirlpool. The pattern exists in the organised relationships between the parts.

Nothing supernatural appears. Local mechanisms produce global behaviour by changing which stable states and coordinated processes the system can sustain. Gradual improvements may therefore produce apparently sudden abilities.

Size alone guarantees nothing. Disconnected calculators have additive scale but little integration. Useful organisation requires communication, shared representations, feedback, learning, and selection among possible interactions.

5. Where time and scale meet

SNN dynamics enlarge the kinds of computation available through time. Network scale enlarges the kinds of organisation available across components. Time enables memory, temporal codes, and collective dynamics. Integrative scale enables abstraction, composition, and coherent internal models.

These are different expansions. A temporally rich system may still be too small or poorly organised to construct useful abstractions. A large system may contain immense capacity while lacking useful temporal dynamics or integration. Neither dimension alone guarantees intelligence.

Their intersection is highly suggestive. A system that remembers its past, represents information through evolving dynamics, combines reusable abstractions, and coordinates them into predictions and plans possesses many of the ingredients of intelligence itself. Intelligence may arise not from time or scale separately, but from organised dynamics operating across both.

6. The holy grail

The holy grail of SNN research is a scalable, continually learning dynamical system whose temporal activity is simultaneously its computation, memory, communication, and learning substrate.

It would learn online without a clean division between training and inference. Local events would contribute to consequences arriving across many timescales. New learning would accumulate without repeatedly erasing old abilities. Oscillations, synchronisation, attractors, metastability, adaptation, and plasticity would perform useful computation rather than merely imitate scalar ANN activations.

The obstacle is learning. Local plasticity knows what happened at one synapse, while intelligent behaviour depends on coordinated consequences across the system and across time. Backpropagation solves this with stored trajectories and global gradients. A continuously operating SNN instead needs local rules that remain scalable while receiving context to assign credit, construct abstractions, and preserve stability.

Efficiency is the conservative promise. Sparse events and neuromorphic hardware may move the practical frontier for always-on agents, asynchronous sensors, adaptive robots, and other systems limited by energy, communication, or latency. The ambitious promise is a machine whose dynamics provide useful computational primitives absent from the forward-pass paradigm.

7. The brain as a search-space prior

The design space spans dynamics, topology, temporal codes, plasticity, credit assignment, timescales, homeostasis, memory, routing, embodiment, and hardware. Blind search is hopeless. This is where biological precedent becomes scientifically valuable.

The brain narrows this space in three ways. It is an existence proof that event-driven components, recurrent state, local plasticity, sparse communication, and modest power can support adaptive intelligence. It offers candidate primitives, including excitation-inhibition balance, oscillations, neuromodulation, eligibility traces, replay, homeostasis, and structural plasticity. It also supplies a search-space prior: evidence that some families of mechanisms are viable enough to investigate first.

Biological fidelity is not the objective. Evolution optimised survival through biological constraints and historical compromises. Some details may be computational discoveries; others may be baggage.

Brain inspiration should therefore generate hypotheses rather than commandments. Each mechanism should be translated into a proposed computational function, reduced to a minimal engineered abstraction, and tested under controlled conditions. The question is not whether a machine resembles a brain. The question is whether a biological mechanism solves an identifiable engineering problem.

We study the brain because evolution has already searched part of the space of continually learning dynamical systems. The goal is to retain what generalises and build beyond evolution.

8. PING as a candidate computational primitive

Pyramidal-interneuron network gamma, or PING, is one concrete candidate for the kind of dynamical machinery this programme seeks. Excitatory activity recruits inhibition; inhibition suppresses excitation; synaptic decay releases the excitatory population; and the cycle repeats. The resulting rhythm belongs to the interacting population. No single neuron contains it.

PING therefore connects temporal computation, integrative scale, and emergence. The recurrent excitatory-inhibitory loop creates structured windows in which neurons are more or less able to fire. Inputs may consequently be treated differently according to when they arrive within the cycle, not only according to their magnitude. As the coordinated population grows, the rhythm may become a shared temporal reference for distributed activity.

This suggests several possible computational roles. PING may gate when signals pass between populations, coordinate otherwise separate pathways, impose sparse windows for activity, organise competition between representations, stabilise recurrent state, or separate phases of communication and plasticity. In these hypotheses, oscillation is not merely an observable side effect. It is internal scheduling machinery generated by the network itself.

Pinglab turns those possibilities into an engineering question. The biological mechanism is reciprocal excitation and inhibition. The computational hypothesis is that the resulting rhythm controls activity, communication, or learning. The engineered abstraction is a trainable excitatory-inhibitory spiking network in which PING dynamics can be manipulated. The controlled test compares matched networks while measuring sparsity, task performance, robustness, routing, or adaptation.

The distinction between correlation and function is decisive. A rhythm can be a useful primitive, an incidental by-product, a stabilising mechanism, or an expensive metronome. Producing gamma does not demonstrate that gamma computes. A functional claim requires an intervention: change or remove the rhythm while preserving relevant alternatives, then measure whether the proposed capability changes with it.

PING is therefore not the manifesto's conclusion. It is a candidate answer to its central question: which collective dynamics become useful computational machinery when temporal systems are trained and scaled? If PING provides reusable gating, coordination, sparsity, memory, or learning functions, it becomes evidence for a machine organised through endogenous dynamics rather than a sequence of externally scheduled forward passes. If it does not, that negative result is equally important. Biological inspiration earns its place only by surviving controlled engineering tests.

The experiment matters because it tests whether an emergent rhythm becomes a controllable and transferable unit of computation itself.